

PSE-TRAVERSE-TPT-MTL-04

Topological Panoptic Triangulation and Möbius-Tripolar Micro-Lift
Eigenständige formale Implementierungsspezifikation v0.4

Sebastian Klemm

Mai 2026

Zusammenfassung

Diese Spezifikation definiert eine implementierungsnahe Erweiterung fuer `pse-traverse`, die auf dem bereits vorhandenen PSE-, Traversal-, Projection-, Cognition-, Evaluation-, PhaseMatrix- und Dual-Fabric-Stitch-Stand aufsetzt. Das Ziel ist ein topologisches Orientierungs- und Triangulationsprofil, das panoptische Kognitionszustände in triangulierbare 5D-Phasenraumfenster überfuehrt, MeshHolo-Artefakte deterministisch evolviert, lokale Möbius-tripolare Mikro-Fasern erzeugt, topologische Merkmale in Claims, Horizonte und Gate-Signale rueckinterpretiert und nur gate-bestandene Kandidaten als auditable Bundles materialisiert. Das Dokument ist eigenständig: Es definiert Scope, Typen, Operatoren, Gates, Reports, CLI, Tests, Definition of Done und Coding-Agent-Auftrag ohne externe Quellenkenntnis vorauszusetzen.

Inhaltsverzeichnis

1	Status, Zweck und harte Architekturregel	2
1.1	Status	2
1.2	Zweck	2
1.3	Nicht-Ziele	2
2	Systemposition im PSE-Stack	3
2.1	Schichtmodell	3
2.2	Eingangs- und Ausgangsvertrag	3
3	Kristallisierte Invarianten	3
4	Normative Kernobjekte	4
4.1	Primitive	4
4.2	TptMtlRunDescriptor	4
4.3	AxisBridge	5
5	PhaseSpaceWindow und Punktmodell	5
5.1	PhaseSpaceWindow	5
5.2	TptPoint	5
6	CarrierContext und tri-temporale Ordnung	6
6.1	CarrierContext	6
6.2	CarrierGate	6
7	MeshHolo und topologische Triangulation	6
7.1	MeshHolo	6
7.2	Kostenfunktion	7
7.3	TopologyGuard	7

8	Möbius-tripolarer Mikro-Lift	7
8.1	Rolle	7
8.2	Subwindow	7
8.3	MicroFiber	7
8.4	Standard-Dualisierung	8
8.5	Seam-Komponente	8
9	Operatoren und Laufzeitpipeline	8
9.1	Operatorfamilie	8
9.2	Referenzpipeline	9
10	Gate-Familie und Outcome-Maschine	9
10.1	Gate-Schema	9
10.2	Normative Gates	10
10.3	Outcome	10
11	Rueckinterpretation in panoptische Kognition	10
11.1	Feature-Mapping	10
11.2	ReinterpretationReport	11
12	Artefakte und Datenmodelle	11
12.1	TopologicalCrystalCandidate	11
12.2	TptMtlBundle	11
12.3	Minimales JSON-Schema	12
13	Modulstruktur und Feature Flags	12
13.1	Referenzmodule	12
13.2	Feature Flags	13
14	Reference CLI	13
15	Konformitaetsklassen	14
16	Testplan	14
16.1	Unit Tests	14
16.2	Integration Tests	14
16.3	Negative Tests	15
17	Evaluationsmetriken	15
18	Definition of Done	15
19	Coding-Agent-Auftrag	16
20	Schlussformel	17

1 Status, Zweck und harte Architekturregel

1.1 Status

Dieses Dokument ist eine normative Spezifikation fuer einen additiven Implementierungsschritt nach PHASEMATRIX-HIVEMIND-03.1. Die Schluesselwoerter **MUST**, **MUST NOT**, **SHOULD** und **MAY** sind verbindlich. Eine Implementierung darf Konformitaet nur fuer jene Klasse beanspruchen, deren Pflichttypen, Gates, Artefakte und Tests vollstaendig implementiert sind.

1.2 Zweck

Der Layer TPT-MTL macht aus panoptischer Kognition eine topologisch pruefbare Orientierungsschicht. Er nimmt bestehende Zustands-, Horizon-, Constraint-, Cell-, Tensor- und Gate-Reports entgegen und erzeugt daraus:

- ein `PhaseSpaceWindow` als kleinste triangulierbare Einheit;
- eine deklarierte `AxisBridge` zwischen semantischer 5D-Codematrix, operativer PSE-Koordinate und Carrier-Metadaten;
- ein `MeshHolo` mit Vertices, Edges, Simplizes, Invarianten, Proof und Trace;
- lokale `MicroFiber`-Objekte aus Primaerphase, Antiphase und Seam-Komponente;
- einen `TopologicalCrystalCandidate`, der erst nach Gates materialisierbar ist;
- ein replayfaehiges `TptMtlBundle`.

Harte Architekturregel

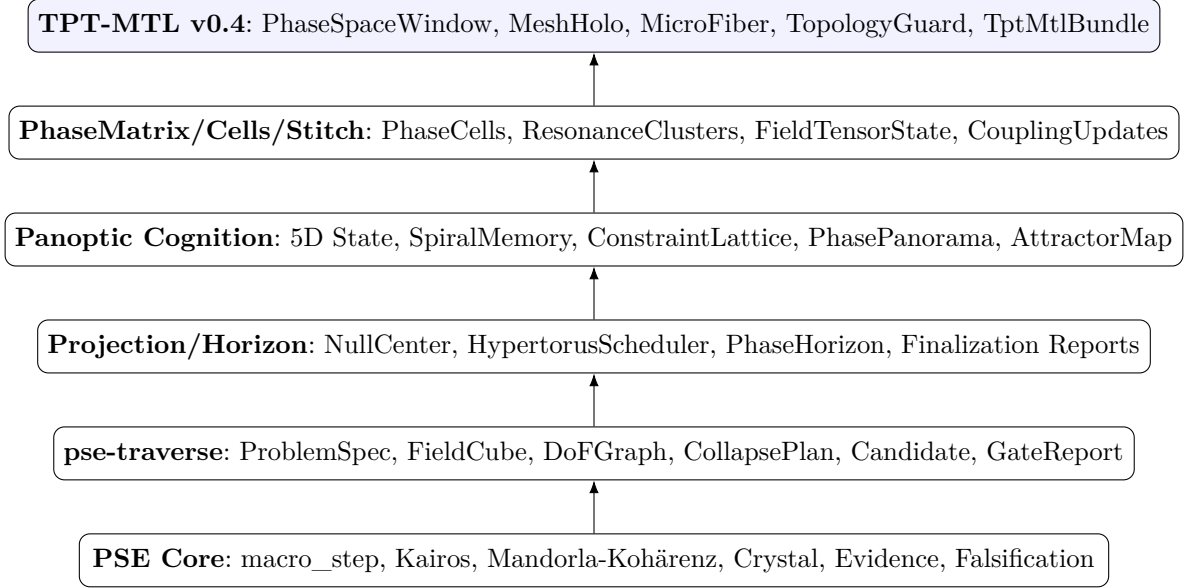
Topologie liefert Struktur-Evidenz, nicht Wahrheit. Kein Mesh, keine Persistenzsignatur, keine Symmetrie und keine Seam-Kohärenz darf `TruthGate`, `BoundaryGate`, `ReplayGate`, `MatrixGate` oder `EmissionGate` ersetzen.

1.3 Nicht-Ziele

TPT-MTL ist kein Ersatz fuer den PSE-Commit-Kern, kein autonomes Aktionssystem, kein Wahrheitsorakel, kein Ueberwachungssystem und keine physikalische Ontologie. Der Layer erzeugt interne Orientierungs-, Proof-, Diagnose- und Handoff-Artefakte. Externe Handlung erfordert ein separates Domain-Handoff mit eigenen Gates.

2 Systemposition im PSE-Stack

2.1 Schichtmodell



2.2 Eingangs- und Ausgangsvertrag

TPT-MTL **MUST** vorhandene PSE-Artefakte referenzieren, aber **MUST NOT** selbst **SemanticCrystal** erzeugen. Der Layer erzeugt Kandidaten und Bundles, die durch bestehende Commit- oder Projection-Bruecken weiterverarbeitet werden koennen.

Inputs:
 CognitionReport | PhasePanorama | ConstraintLattice | CandidateBundle
 FieldTensorState | ResonanceFabricState | StitcherReport
 MatrixBoundaryReport | TruthMaintenanceReport | EvidenceRefs

Outputs:
 PhaseSpaceWindow
 MeshHolo
 MicroFiberSet
 TopologicalCrystalCandidate
 TptMtlGateReport
 TptMtlBundle
 ReplayManifest

3 Kristallisierte Invarianten

ID	Normative Invariante
I-01	State before topology. Kein triangulierbares Fenster ohne gueltigen, typisierten, normalisierten, admissiblen und tracebaren Zustand.
I-02	Window before point. Ein isolierter Punkt ist kein hinreichender Traeger. Die kleinste triangulierbare Einheit ist ein PhaseSpaceWindow .
I-03	Axis separation. Semantische Achsen, operative Koordinaten und Carrier-Metadaten MUST getrennt deklariert werden.
I-04	Topology before claim, gate before emission. Topologische Features werden erst in Claim-Kandidaten rueckinterpretiert und danach gegated.

ID	Normative Invariante
I-05	Ephemeral exploration, persistent proof. Explorative Mesh-/Mikro-Faser-Zustände dürfen kompaktieren; Trace, Evidence, Gate-History, Blockadegründe und Proofs dürfen nicht verschwinden.
I-06	No stateful null center. Der Null-Center ist Ordnungsanker und MUST NOT als datenhaltiger Vertex, Speicher oder dritter Zustand materialisiert werden.
I-07	Antiphase by proof. Eine Antiphase MUST aus einer deklarierten Dualisierungsregel mit Proof, Parametern und Digest entstehen.
I-08	TopologyGuard on mutation. Jede Mesh-Mutation MUST einen TopologyGuardProof erzeugen.
I-09	Budgeted recursion. Rekursive Verfeinerung MUST durch Depth-, Step-, Vertex-, Simplex-, Time- und Memory-Budgets begrenzt sein.
I-10	Replay identity. Identische Inputs, RD, Policies und Operatorversionen MUST identische Digests für Window, Mesh, MicroFibers, GateReport und Bundle erzeugen.

4 Normative Kernobjekte

4.1 Primitive

```
pub type Hash256 = [u8; 32];
pub type Fixed = CanonicalNumber;
pub type RationalFixed = CanonicalRational;

pub struct EvidenceRef {
  pub evidence_id: Hash256,
  pub kind: String,
  pub source_digest: Hash256,
  pub trace_ref: Hash256,
}
```

4.2 TptMtlRunDescriptor

```
pub struct TptMtlRunDescriptor {
  pub rdver: String,           // "TPT-MTL-0.4"
  pub seed: String,           // deterministic only; no entropy claim
  pub canonicalization: String, // e.g. "jcs-rfc8785"
  pub numeric_policy: NumericPolicy,
  pub tie_break: TieBreakPolicy,
  pub domain_profile: String,
  pub source_boundaries: Vec<Hash256>,
  pub operator_versions: BTreeMap<String, String>,
  pub budgets: TptMtlBudgets,
  pub axis_policy: AxisPolicy,
  pub mesh_policy: MeshPolicy,
  pub micro_lift_policy: MicroLiftPolicy,
  pub gate_thresholds: TptMtlThresholds,
}

pub enum NumericPolicy { Fixed, Rational, BoundedApprox { tolerance: Fixed } }
pub enum TieBreakPolicy { DigestLexicographic, StableIndexThenDigest }
```

4.3 AxisBridge

Die `AxisBridge` trennt drei Achsenebenen. Eine Implementierung **MUST** fuer jede Koordinate deren Herkunft deklarieren.

```
pub struct AxisPolicy {
  pub semantic_axes: [String; 5],    // R,F,T,S,E or profile-defined equivalents
  pub runtime_axes: [String; 5],     // psi,rho,omega,chi,tau
  pub carrier_axes: Vec<String>,     // n0,t2,t1,alpha,beta,mandorla,lambda
  pub translation_policy: String,
}

pub struct AxisBridgeReport {
  pub report_id: Hash256,
  pub semantic_digest: Hash256,
  pub runtime_digest: Hash256,
  pub carrier_digest: Hash256,
  pub translation_proofs: Vec<Hash256>,
  pub violations: Vec<String>,
  pub passed: bool,
}
```

5 PhaseSpaceWindow und Punktmodell

5.1 PhaseSpaceWindow

```
pub struct PhaseSpaceWindow {
  pub window_id: Hash256,
  pub domain_profile: String,
  pub input_refs: Vec<Hash256>,
  pub points: Vec<TptPoint>,
  pub carrier: CarrierContext,
  pub horizon_refs: Vec<Hash256>,
  pub constraint_refs: Vec<Hash256>,
  pub boundary_ref: Hash256,
  pub sampling_policy_hash: Hash256,
  pub trace_ref: Hash256,
  pub rd_hash: Hash256,
}
```

5.2 TptPoint

```
pub struct TptPoint {
  pub point_id: Hash256,
  pub x: [Fixed; 5],
  pub meta: PointMeta,
  pub provenance: Vec<EvidenceRef>,
  pub carrier_ref: Hash256,
  pub gate_refs: Vec<Hash256>,
}

pub struct PointMeta {
  pub semantic_axis_labels: [String; 5],
  pub runtime_axis_labels: [String; 5],
  pub status: PointStatus,
}
```

```
pub enum PointStatus { Active, NoiseCandidate, Blocked, Latent }
```

Provenienzpflcht

Jeder `TptPoint` **MUST** mindestens eine `EvidenceRef` mit `SourceDigest`, `SegmentRef` oder `StateRef` tragen. Punkte ohne Provenienz **MUST** als `NoiseCandidate` protokolliert und aus gate-relevanter Triangulation ausgeschlossen werden.

6 CarrierContext und tri-temporale Ordnung

6.1 CarrierContext

```
pub struct CarrierContext {
  pub carrier_id: Hash256,
  pub null_center_ref: Hash256,          // stateless anchor; not a vertex
  pub intrinsic_time: RationalFixed,    // t2: evolution
  pub extrinsic_time: u64,              // t1: commit index
  pub helix_alpha_ref: Hash256,
  pub helix_beta_ref: Hash256,
  pub mandorla_ref: Hash256,
  pub active_ladder_phase: String,
  pub ladder_refs: Vec<Hash256>,
  pub metrics_hash: Hash256,
  pub trace_ref: Hash256,
}
```

6.2 CarrierGate

`CarrierGate` besteht genau dann, wenn `null_center_ref` nicht als `Vertex` materialisiert wird, intrinsische und extrinsische Zeit getrennt bleiben, Helix- und Mandorla-Referenzen vorhanden sind, Ladder-Migration tracebar ist und keine `Boundary` verletzt wird.

$$G_{carrier} = NC_{stateless} \wedge TimeSep(t_2, t_1) \wedge HelixOk \wedge MandorlaOk \wedge TraceReady \wedge BoundaryPass. \quad (1)$$

7 MeshHolo und topologische Triangulation

7.1 MeshHolo

```
pub struct MeshHolo {
  pub mesh_id: Hash256,
  pub window_ref: Hash256,
  pub vertices: Vec<MeshVertex>,
  pub edges: Vec<MeshEdge>,
  pub simplices: Vec<Simplex>,
  pub invariants: TopologicalInvariants,
  pub proof: MeshProof,
  pub trace_ref: Hash256,
}

pub struct TopologicalInvariants {
  pub betti: Vec<i64>,
  pub persistence_digest: Hash256,
```

```

    pub entropy_class: EntropyClass,
    pub lambda_gap: Fixed,
}

pub enum EntropyClass { Explore, Refine, Simplify, Freeze }

```

7.2 Kostenfunktion

Fuer eine Kandidatentriangulation T gilt:

$$J(T) = \alpha_1 Sym(T) + \alpha_2 Res(T) + \alpha_3 TopoStab(T) + \alpha_4 Seam(T) + \alpha_5 Carrier(T) - \alpha_6 EntropyPenalty(T) - \alpha_7 Comp(T) \quad (2)$$

Alle Koeffizienten **MUST** im RunDescriptor deklariert werden. Wenn ein Term nicht implementiert ist, **MUST** der Konformitaetslevel entsprechend niedriger sein.

7.3 TopologyGuard

$$G_{topo} = 1 \Leftrightarrow \Delta\beta \in AllowedShift \wedge W_p(PD_{old}, PD_{new}) \leq \theta_{PD} \wedge TraceReady. \quad (3)$$

Eine nicht begruendete Veraenderung von Betti-Vektor, Persistenzdiagramm oder Inzidenzgrenze **MUST** zu TopologyFailure fuehren.

8 Möbius-tripolarer Mikro-Lift

8.1 Rolle

Der Mikro-Lift ist eine lokale Faser-Schicht zwischen PhaseSpaceWindow und MeshHolo. Er verdreifacht nicht blind alle Vertices. In der Mindestklasse MTL-1 bleibt die Vertex-Identitaet erhalten; die Mikro-Faser wird als Descriptor an den Punkt gebunden.

8.2 Subwindow

```

pub struct LocalSubwindow {
    pub subwindow_id: Hash256,
    pub center_point_id: Hash256,
    pub point_ids: Vec<Hash256>,
    pub neighborhood_policy_hash: Hash256,
    pub metric: String,
    pub k_or_radius: String,
    pub trace_ref: Hash256,
}

```

LocalSubwindow **MUST** deterministisch aus Fenster, Punkt, RD und Nachbarschaftspolitik entstehen. Gleichstaende werden ueber TieBreakPolicy aufgeloeset.

8.3 MicroFiber

```

pub struct MicroFiber {
    pub fiber_id: Hash256,
    pub point_id: Hash256,
    pub subwindow_ref: Hash256,
    pub primary: PrimaryPhase,
    pub dual: DualAntiphase,
    pub seam: SeamComponent,
    pub carrier_ref: Hash256,
    pub lift_proof: Hash256,
}

```



```

    pub trace_ref: Hash256,
}

pub struct PrimaryPhase {
    pub vector: [Fixed; 5],
    pub local_features_hash: Hash256,
    pub digest: Hash256,
}

pub struct DualAntiphase {
    pub vector: [Fixed; 5],
    pub rule_id: String,
    pub proof_ref: Hash256,
    pub digest: Hash256,
}

pub struct SeamComponent {
    pub vector: [Fixed; 5],
    pub proof_ref: Hash256,
    pub carrier_coherence: Fixed,
    pub digest: Hash256,
}

```

8.4 Standard-Dualisierung

Die minimale Standardregel MTL-D1 spiegelt den Punkt an der lokalen Gegenrichtung relativ zum Subwindow-Zentrum. Bei Degeneration **MUST** ein typisiertes Failure-Objekt entstehen.

$$D_i = I - 2 \frac{v_i v_i^\top}{\|v_i\|^2 + \epsilon}, \quad v_i = x_i - \bar{x}_{U_i}, \quad x_i^- = \text{clip}_{[0,1]^5} (\bar{x}_{U_i} + D_i(x_i - \bar{x}_{U_i})). \quad (4)$$

8.5 Seam-Komponente

$$z_i = [x_i^+, x_i^-, x_i^+ \odot x_i^-, |x_i^+ - x_i^-|], \quad m_i = \sigma(A_{RD} z_i + B_{RD} \phi(U_i, K)). \quad (5)$$

`m_i` ist gueltig genau dann, wenn es bounded, symmetrie-zulaessig, carrier-kohärent und tracebar ist.

9 Operatoren und Laufzeitpipeline

9.1 Operatorfamilie

Operator	Typ	Semantik
BuildWindow	$\text{State} \rightarrow \text{Window}$	Erzeugt <code>PhaseSpaceWindow</code> aus <code>Cognition-/Matrix-State</code> .
AxisBridge	$\text{Window} \rightarrow \text{Report}$	Deklariert und prueft Achsenherkunft.
SeedMesh	$\text{Window} \rightarrow \text{Mesh}$	Erzeugt deterministisches initiales Mesh.
EvolveMesh	$\text{Mesh} \rightarrow \text{Mesh}$	Lokale Mesh-Mutationen unter <code>TopologyGuard</code> .
LiftMicro	$\text{Window} \rightarrow \text{Fibers}$	Erzeugt Subwindows, Primaerphasen, Antiphasen und Seams.
WeightMicroMesh	$\text{Mesh} \times \text{Fibers} \rightarrow \text{Mesh}$	Integriert Mikro-Faser-Signale in Kanten- und Simplexkosten.
RefineRecursive	$\text{Mesh} \rightarrow \text{Mesh}$	DK/SW/PI/WT/Press unter Budget.

Operator	Typ	Semantik
EvaluateCarrier	Mesh $\times$ Carrier $\rightarrow$ Report	Prueft Carrier-Kontinuitaet, Kairos und Migration.
Reinterpret	Mesh $\rightarrow$ Claims/Horizons	Uebersetzt Features in ClaimCandidates und HorizonUpdates.
GateAll	Candidate $\rightarrow$ Decision	Fuehrt fail-closed Gate-Familie aus.
Bundle	Reports $\rightarrow$ Bundle	Materialisiert replayfaehiges Artefaktbuendel.

9.2 Referenzpipeline

```

fn run_tpt_mtl(input: TptMtlInput, rd: TptMtlRunDescriptor) -> TptMtlOutcome {
  let state = load_and_validate_state(input, rd)?;
  let window = build_phase_space_window(state, rd);
  if !gate_adapter(&window, rd) { return Recalibrate(adapter_report(window)); }

  let axis = build_axis_bridge_report(&window, rd);
  if !axis.passed { return Recalibrate(axis.into_report()); }

  let mesh0 = seed_mesh_holo(&window, rd);
  let mesh1 = evolve_mesh(mesh0, rd); // guarded local moves
  if !gate_topology(&mesh1, rd) { return Hold(mesh1.failure_report()); }

  let fibers = lift_micro_fibers(&window, rd);
  if !gate_micro_lift(&fibers, rd) { return Hold(fibers.gate_report()); }

  let mesh2 = weight_mesh_with_microfibers(mesh1, fibers, rd);
  let refined = refine_recursive(mesh2, rd); // budgeted
  let carrier = evaluate_carrier(&refined, &window.carrier, rd);

  let interp = reinterpret_mesh_to_panoptic(&refined, &carrier, rd);
  let gate_report = gate_all(&interp, &refined, &carrier, rd);
  let candidate = form_topological_crystal_candidate(&interp, &refined, &gate_report, rd);

  if gate_report.matrix && gate_report.emission {
    Emit(bundle(candidate, gate_report, rd))
  } else {
    Hold(bundle(candidate, gate_report, rd))
  }
}

```

10 Gate-Familie und Outcome-Maschine

10.1 Gate-Schema

```

pub struct GateResult {
  pub gate_id: String,
  pub passed: bool,
  pub diagnostics: Vec<String>,
  pub evidence_refs: Vec<EvidenceRef>,
  pub on_fail: FailurePolicy,
  pub trace_ref: Hash256,
}

```

```
}
pub enum FailurePolicy { Hold, Degrade, Recalibrate, Quarantine, Abort }
```

10.2 Normative Gates

Gate	Bestehensbedingung
AdapterGate	Window gueltig, Punkte provenienztragend oder NoiseCandidate, Sampling determiniert.
AxisGate	Achsenherkunft deklariert; keine Vermischung von semantischen, operativen und Carrier-Achsen ohne TranslationProof.
SymmetryGate	deklarierte Symmetrie eingehalten oder SymmetryBreak mit Proof und Policy dokumentiert.
TopologyGate	Betti/Persistenz stabil oder allowed shift; TopologyGuardProof vorhanden.
EntropyGate	Entropieklasse berechnet; Explore/Refine/Simplify/Freeze-Entscheidung dokumentiert.
MicroLiftGate	Subwindows, Primaerphase, Antiphase, Seam und Proofs vollstaendig; keine Punkt-Lifts ohne Provenienz.
CarrierGate	Null-Center stateless; Zeiten getrennt; Mandorla/Helix/Ladder tracebar.
KairosGate	Deformation, Kohärenz, Resonanz, Fokus, Naht, Kristallreife und Topologie bestehen.
TruthGate	Rueckinterpretierte Claims haben Status, Scope, Evidence, Lineage und geloeste Widersprueche.
BoundaryGate	Daten-, Domain-, Sicherheits-, Aktions- und Scope-Grenzen eingehalten.
ReplayGate	Wiederholung erzeugt identische Digests oder deklarierte Toleranzklasse.
MatrixGate	alle vorgelagerten Gates bestanden; keine unresolved critical contradictions; ProofObligations registriert.
EmissionGate	Output im erlaubten Scope; vollstaendige Provenienz; keine externe Handlung.

10.3 Outcome

$$G_{all} = G_{adapter} \wedge G_{axis} \wedge G_{topo} \wedge G_{micro} \wedge G_{carrier} \wedge G_{truth} \wedge G_{boundary} \wedge G_{replay} \wedge G_{matrix} \wedge G_{emission}. \quad (6)$$

$$Decision(x) = \begin{cases} Emit(x), & G_{all} = 1, \\ Abort(x), & \text{harte Boundary oder Replay-Identitaet verletzt,} \\ Quarantine(x), & \text{offener kritischer Claim als Praemisse wirken wuerde,} \\ Recalibrate(x), & \text{Adapter-, Achsen-, Faser- oder Carrierpolitik unzureichend ist,} \\ Hold(x), & \text{sonst.} \end{cases} \quad (7)$$

11 Rueckinterpretation in panoptische Kognition

11.1 Feature-Mapping

Topologisches Feature	Rueckinterpretation	Pflichtstatus
Persistente Komponente	robuste Motivfamilie, Cluster, zusammenhaengender Pfadraum	Derived nach Evidence
Kurzlebiges Feature	Rauschen, lokale Artefaktkante, instabile Hypothese	Open/Noise
Persistente Schleife	zyklische Abhaengigkeit, Wiederkehr, unaufgeloeste Relation	ClaimCandidate mit Contradiction-Check
Hohlraum/Luecke	fehlender Pfad, Blockade, Datenluecke, nicht erklarte Boundary	Counter-Horizon
Betti-Shift	Regimewechsel, Deformation, Migrationserfordernis	Gate-relevant
Hohe Entropiezone	Explorations- oder Refinementzone	nicht emissionsfaehig
Niedrige Entropiezone	Verdichtungs- oder Capturezone	Candidate moeglich

11.2 ReinterpretationReport

```
pub struct ReinterpretationReport {
  pub report_id: Hash256,
  pub mesh_ref: Hash256,
  pub claim_candidates: Vec<Hash256>,
  pub current_horizon_updates: Vec<Hash256>,
  pub counter_horizon_updates: Vec<Hash256>,
  pub gate_signals: Vec<Hash256>,
  pub proof_obligations: Vec<Hash256>,
  pub contradictions: Vec<Hash256>,
  pub trace_ref: Hash256,
}
```

Topologische Luecken, simulierte Pfade und future-window Features **MUST** als counterfactual, latent oder blocked markiert bleiben. Sie **MUST NOT** als beobachtete Claims emittiert werden.

12 Artefakte und Datenmodelle

12.1 TopologicalCrystalCandidate

```
pub struct TopologicalCrystalCandidate {
  pub candidate_id: Hash256,
  pub window_ref: Hash256,
  pub mesh_ref: Hash256,
  pub micro_fiber_set_ref: Hash256,
  pub carrier_report_ref: Hash256,
  pub reinterpretation_report_ref: Hash256,
  pub gate_report_ref: Hash256,
  pub evidence_refs: Vec<EvidenceRef>,
  pub status: CandidateStatus,
  pub trace_ref: Hash256,
}

pub enum CandidateStatus { Draft, Hold, Quarantined, GatePassed, Emitted }
```

12.2 TptMtlBundle

```
pub struct TptMtlBundle {
    pub bundle_id: Hash256,
    pub rd_hash: Hash256,
    pub source_manifest_hash: Hash256,
    pub phase_space_window_hash: Hash256,
    pub axis_bridge_report_hash: Hash256,
    pub mesh_holo_hash: Hash256,
    pub micro_fiber_set_hash: Hash256,
    pub carrier_report_hash: Hash256,
    pub reinterpretation_report_hash: Hash256,
    pub gate_report_hash: Hash256,
    pub crystal_candidate_hash: Hash256,
    pub trace_hash: Hash256,
    pub replay_manifest_hash: Hash256,
}
```

12.3 Minimales JSON-Schema

```
{
  "rdver": "TPT-MTL-0.4",
  "run_descriptor": "sha256:...",
  "phase_space_window": "sha256:...",
  "axis_bridge_report": "sha256:...",
  "mesh_holo": {
    "vertices_digest": "sha256:...",
    "edges_digest": "sha256:...",
    "simplices_digest": "sha256:...",
    "invariants": {
      "betty": [0,0,0,0,0],
      "persistence_digest": "sha256:...",
      "entropy_class": "Explore|Refine|Simplify|Freeze",
      "lambda_gap": "fixed"
    },
    "proof": "sha256:..."
  },
  "micro_fibers_digest": "sha256:...",
  "carrier_report": "sha256:...",
  "reinterpretation_report": "sha256:...",
  "gate_report": "sha256:...",
  "crystal_candidate": "sha256:...",
  "outcome": "Emit|Hold|Degrade|Recalibrate|Quarantine|Abort",
  "trace": "sha256:...",
  "replay_manifest": "sha256:..."
}
```

13 Modulstruktur und Feature Flags

13.1 Referenzmodule

```
crates/pse-traverse/src/topology/
mod.rs
primitives.rs
run_descriptor.rs
axis_bridge.rs
phase_window.rs
carrier.rs
```

```

mesh_holo.rs
persistence.rs
topology_guard.rs
micro_lift.rs
subwindow.rs
primary_phase.rs
dual_antiphase.rs
seam.rs
micro_fiber.rs
mesh_weights.rs
refine.rs
reinterpret.rs
gates.rs
artifact.rs
replay.rs
pipeline.rs

tools/pse-traverse-topology-cli/
src/main.rs

```

13.2 Feature Flags

Flag	Bedeutung
topology	Aktiviert Basistypen, PhaseSpaceWindow, MeshHolo und Gates.
topology-cli	Baut CLI-Tool.
topology-mtl	Aktiviert MicroFiber, MTL-Dualisierung und Seam-Komponenten.
topology-refine	Aktiviert rekursive DK/SW/PI/WT/Press-Verfeinerung.
topology-carrier	Aktiviert CarrierContext, KairosGate und CarrierMigration-Reports.
topology-reinterpret	Aktiviert Rueckinterpretation in Claims/Horizonte/Gate-Signale.
topology-handoff	Aktiviert Handoff-Kandidaten ohne externen Commit.

14 Reference CLI

```

pse-traverse-topology inspect --input state.json
pse-traverse-topology window --state state.json --rd rd.json --out window.json
pse-traverse-topology axis --window window.json --out axis_report.json
pse-traverse-topology triangulate --window window.json --out mesh_holo.json
pse-traverse-topology lift --window window.json --out micro_fibers.json
pse-traverse-topology weight --mesh mesh_holo.json --fibers micro_fibers.json --out
  mesh_weighted.json
pse-traverse-topology refine --mesh mesh_weighted.json --depth 3 --out refined_mesh.json
pse-traverse-topology carrier --mesh refined_mesh.json --window window.json --out
  carrier_report.json
pse-traverse-topology reinterpret --mesh refined_mesh.json --carrier carrier_report.json
  --out reinterpretation.json
pse-traverse-topology gate --window window.json --mesh refined_mesh.json --fibers
  micro_fibers.json --reinterpretation reinterpretation.json --out gate_report.json
pse-traverse-topology bundle --gate gate_report.json --out tpt_mtl_bundle.json
pse-traverse-topology replay --bundle tpt_mtl_bundle.json --expect same-digest
pse-traverse-topology verify --bundle tpt_mtl_bundle.json

```

15 Konformitätsklassen

Klasse	Anforderungen
TPTM-0	RD, Primitive, Canonicalization, Digest, Trace, PhaseSpaceWindow, AdapterFailure.
TPTM-1	TPTM-0 plus AxisBridge, TptPoint, CarrierContext und AdapterGate.
TPTM-2	TPTM-1 plus MeshHolo, SeedMesh, SymmetryGate, TopologyGate, EntropyGate.
TPTM-3	TPTM-2 plus impliziter MicroFiber-Lift, MTL-D1, SeamComponent, MicroLiftGate.
TPTM-4	TPTM-3 plus MicroFiber-Gewichtung im Mesh und rekursive Refinement-Operatoren unter Budget.
TPTM-5	TPTM-4 plus CarrierGate, KairosGate, ReinterpretationReport, TopologicalCrystalCandidate, TptMtlBundle, Replay.
TPTM-6	TPTM-5 plus optionale Integration mit PhaseMatrix-Federation, ADAMANT-kompatiblen ProofObligations und test-backed Domain Profiles.

16 Testplan

16.1 Unit Tests

1. canonicalization_is_idempotent
2. axis_bridge_rejects_mixed_axis_origin
3. phase_window_requires_provenance_or_noise_status
4. null_center_is_not_materialized_as_vertex
5. mtl_d1_dualization_is_deterministic
6. seam_requires_carrier_context
7. topology_guard_rejects_unallowed_betti_shift
8. entropy_classification_is_total
9. recursive_refinement_respects_budget
10. gate_unknown_status_fails_closed

16.2 Integration Tests

1. Minimaler Cognition/PhaseMatrix-State erzeugt PhaseSpaceWindow und MeshHolo.
2. PhaseSpaceWindow -> MeshHolo -> MicroFiberSet -> TptMtlBundle ist End-to-End lauffähig.
3. Identische Inputs erzeugen byte-identische Bundles.
4. TopologyGuard-Failure führt zu Hold oder Quarantine, niemals Emit.
5. Boundary-Failure führt zu Abort.
6. Gate-bestandener Kandidat erzeugt TopologicalCrystalCandidate mit vollständiger Provenienz.
7. Reinterpretation erzeugt Current- und Counter-Horizon-Updates mit Claim-Status.

16.3 Negative Tests

1. Punkt ohne Provenienz wird geliftet: **MUST** NoiseCandidate oder Failure sein.
2. Null-Center wird als Vertex materialisiert: **MUST** Abort sein.
3. Antiphase entsteht durch nicht deklarierte Negation: **MUST** DualizationFailure sein.
4. Seam wird ohne Carrier-Kontext berechnet: **MUST** SeamFailure sein.
5. Identischer Input erzeugt verschiedene MicroFiber-Digests: **MUST** ReplayFailure sein.
6. Topologieaenderung ohne AllowedShift: **MUST** TopologyFailure sein.
7. Mesh-Kohärenz wird als Wahrheit emittiert: **MUST** MatrixGateFailure sein.
8. Budget Exhaustion fuehrt zu Emit: **MUST** fehlschlagen.
9. Nicht deklarierte Quelle im RD fehlt: **MUST** BoundaryFailure sein.

17 Evaluationsmetriken

Metrik	Bedeutung
AdapterTotalityRate	Anteil der Inputs, die zu gueltigem Window oder typisiertem Failure fuehren.
AxisBridgeValidity	Anteil der Punkte mit korrekt deklarierter Achsenherkunft.
MeshDeterminismIdentity	Anteil identischer Mesh-Replays.
TopologyRobustness	Stabilitaet von Betti-/Persistenzsignaturen unter erlaubten Perturbationen.
MicroLiftCoverage	Anteil gueltig gelifteter Punkte.
SeamConsistencyRate	Anteil gueltiger Seam-Komponenten.
CarrierContinuity	Erhaltung von Richtung, Trace und Carrier-Trennung.
FalseCrystalRate	Anteil gate-bestandener Kandidaten, die spaeter durch Evidence/Replay widerlegt werden.
TraceCompleteness	Anteil aller Uebergaenge mit vollstaendigem Trace.
ReplayIdentity	Byte- oder Digest-Identitaet kompletter Bundles.

18 Definition of Done

Eine Referenzimplementierung TPTM-5 gilt als abgeschlossen, wenn:

1. alle public types dokumentiert sind;
2. alle identity-bearing Pfade deterministisch sind;
3. keine platform floats im Audit-, Gate- oder Hash-Pfad verwendet werden;
4. PhaseSpaceWindow, AxisBridgeReport, MeshHolo, MicroFiberSet, ReinterpretationReport, TptMtlGateReport und TptMtlBundle kanonisch serialisierbar sind;
5. TopologyGuardProof bei jeder Mesh-Mutation erzeugt wird;
6. MicroFiber niemals ohne Subwindow, Provenienz, DualProof, SeamProof und Trace entsteht;
7. NullCenter niemals als stateful Vertex materialisiert wird;
8. alle Gates fail-closed implementiert sind;
9. Replay fuer mindestens drei Golden Bundles identische Digests erzeugt;

10. CLI-End-to-End von `window` bis `verify` besteht;
11. Unit-, Integration-, Negative- und Replay-Tests bestanden sind;
12. der Layer keine `SemanticCrystal`, keine externen Commit-Artefakte und keine externe Aktion erzeugt.

19 Coding-Agent-Auftrag

PATCH-ID: PSE-TRAVERSE-TPT-MTL-04

TITLE: Topological Panoptic Triangulation and Möbius-Tripolar Micro-Lift

GOAL: Implementiere einen additiven `pse-traverse` Layer, der vorhandene Cognition-, PhaseMatrix-, Cell- und FieldTensor-Reports in deterministische topologische Orientierungsartefakte ueberfuehrt: `PhaseSpaceWindow`, `MeshHolo`, `MicroFiberSet`, `ReinterpretationReport`, `TopologicalCrystalCandidate` und `TptMtlBundle`. Der Layer erzeugt keine PSE-Crystals und keine externen Commits.

MUST

1. Module unter `crates/pse-traverse/src/topology/` gemäss Referenzstruktur anlegen.
2. Feature Flag `topology` und optional `topology-cli`, `topology-mtl`, `topology-refine`, `topology-carrier`, `topology-reinterpret` einrichten.
3. RD, NumericPolicy, AxisPolicy, Budgets, GateThresholds und Operatorversionen implementieren.
4. `PhaseSpaceWindow`, `TptPoint`, `CarrierContext`, `MeshHolo`, `MicroFiber`, `TopologicalCrystalCandidate`, `TptMtlBundle` implementieren.
5. BTreeMap/BTreeSet oder sortierte Vektoren vor Hashing nutzen; JCS oder vorhandene Canonicalization verwenden.
6. Adapter-, Axis-, Symmetry-, Topology-, Entropy-, MicroLift-, Carrier-, Kairos-, Truth-, Boundary-, Replay-, Matrix- und Emission-Gates fail-closed implementieren.
7. CLI-Kommandos gemäss Reference CLI bereitstellen.
8. Golden Fixtures fuer Emit, Hold, Recalibrate, Quarantine, Abort und Replay liefern.

MUST NOT

1. `SemanticCrystal` oder externe Finalisierungsartefakte erzeugen.
2. Topologie, Seam-Kohärenz oder Mesh-Stabilität als Wahrheit behandeln.
3. Null-Center als Vertex, Speicher oder dynamischen Zustand materialisieren.
4. nicht deklarierte Quellen, ambient time oder nichtdeterministische Iterationsordnung im Audit-Pfad verwenden.
5. Budget Exhaustion, Gate-Unklarheit oder fehlende Evidence zu Emit eskalieren.

SHOULD

1. Die Implementierung klein beginnen: erst TPTM-2, danach TPTM-3, dann TPTM-5.
2. Bestehende `pse-traverse` Canonicalization, Report- und Replay-Konventionen wiederverwenden.
3. README und CHANGELOG mit Status, CLI-Beispielen, Feature-Flags und Konformitätsklasse aktualisieren.

20 Schlussformel

$$\Sigma_{cog/matrix} \xrightarrow{BuildWindow} W_{H5,K} \xrightarrow{Triangulate} M_H \xrightarrow{LiftMicro} \{\mu_i\} \xrightarrow{Reinterpret} (Claims, H, \bar{H}, G) \xrightarrow{Gate} Bundle. \quad (8)$$

TPT-MTL erweitert den Horizont der panoptischen Kognition nicht durch mehr Behauptung, sondern durch mehr pruefbare Form. Der Layer macht sichtbar, welche Regionen des Phasenraums stabil, latent, blockiert, widerspruechlich, carrierfaehig, mikro-faser-kohaerent oder kristallisationsreif sind. Persistenz entsteht erst nach Gate-Passage; alles andere bleibt Hold, Recalibrate, Quarantine oder Abort.